

UNIVERSITY OF OSLO

COCOTB

Starting on the right foot



Yngve Hafting
Lecturer
Department of Informatics

2026-03-05

Cocotb cosimulation

How does timing work?

Cosimulation concept
Coroutines and Tasks
Event queue, Triggers and Timing



[https://github.com/yngveha/seminars/
tree/main/2026/inventas/](https://github.com/yngveha/seminars/tree/main/2026/inventas/)



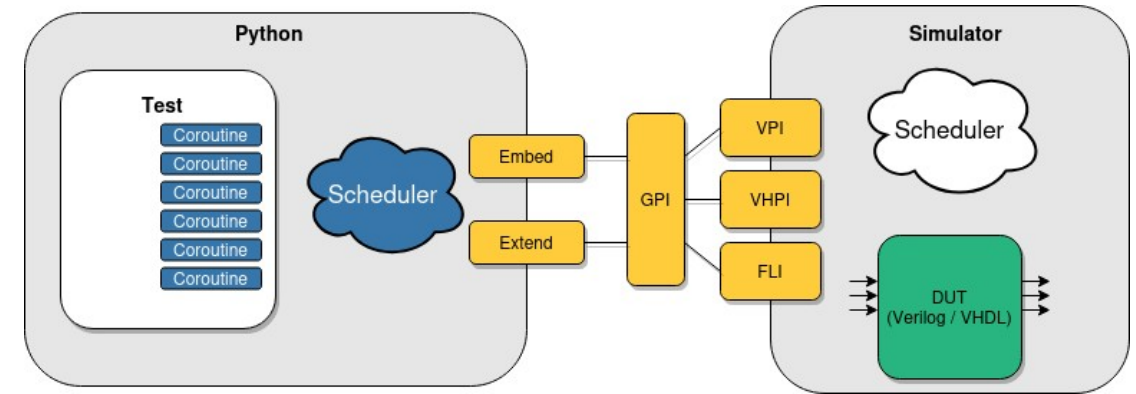
UNIVERSITY
OF OSLO



Cosimulation concept

Cocotb structure

- Cocotb uses ordinary VHDL simulation + a software testbench.
- Cocotb
 - is based on python
 - productive, lots of resources online
 - can interface other software...
 - Requires a single HDL top layer, *DUT*
 - *Can be a VHDL testbench...*
 - Both simulation code and synthesizable code is allowed
 - VHDL testbench code will run alongside python based tests
 - *Context switching* can be managed(...)
- Full simulation hierarchy can be accessed
 - *provided the simulator supports this* through VPI/VHPI/FLI

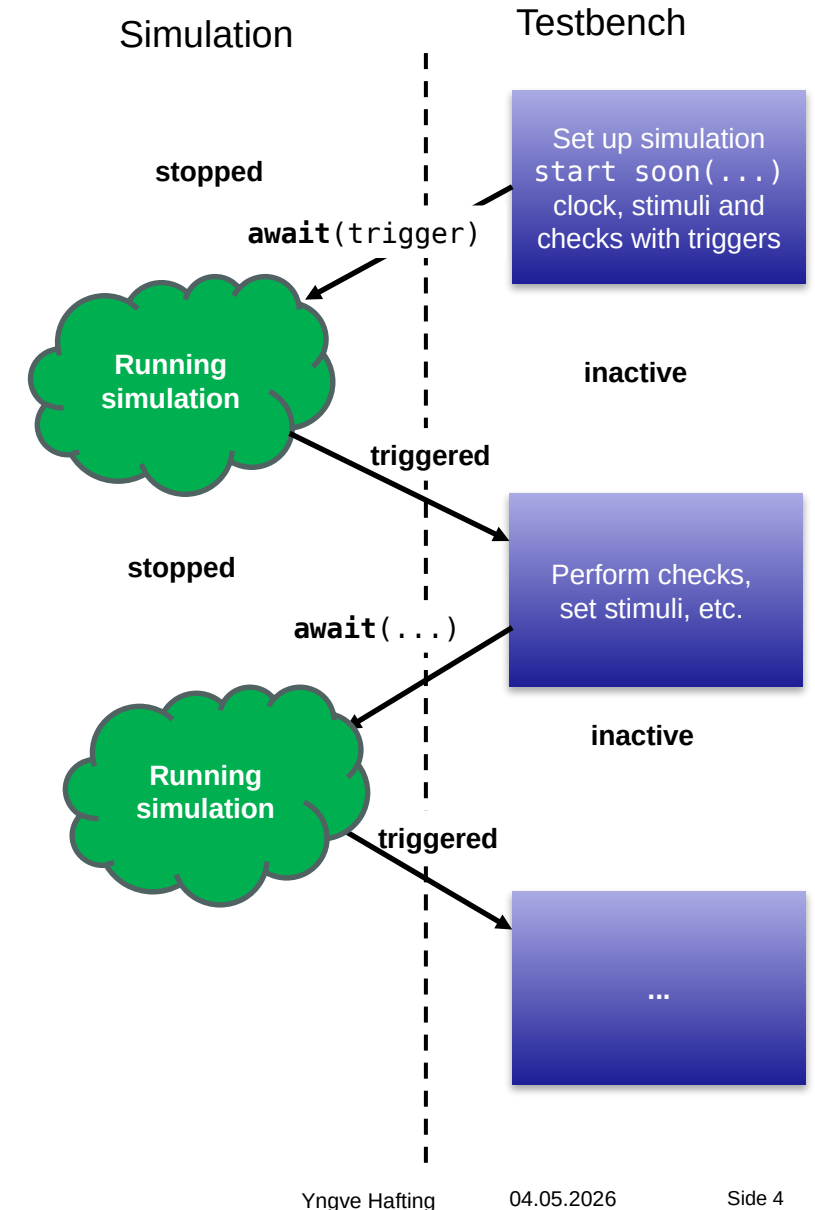


<https://docs.cocotb.org/en/stable/>

Cosimulation concept

Context switching

- When testbench (Python) code is run...
 - Simulation is stopped
- Communication through VPI/VHPI interfaces
 - represented by cocotb "triggers".
- When a trigger is awaited,
 - Simulation runs until triggered =>
 - tb resumes until next await.
- Python code is run in a *single thread*.
 - ♦ Multithreading *can* be used for other tasks (network interfaces etc)



Cosimulation concept

Tasks and coroutines

- *“Coroutines are computer program components that must to be suspended and resumed, generalizing subroutines for cooperative multitasking.”*
<https://en.wikipedia.org/wiki/Coroutine>
- *“Tasks are cocotb objects that wrap coroutines and are used to schedule concurrent execution of the testbench coroutines.”*
<https://docs.cocotb.org/en/stable/coroutines.html>
- Example next page

Cosimulation concept

Tasks and coroutines

- Coroutines are defined using **async def**
- Tasks can be
 - Scheduled using **start_soon** (*non blocking*)
 - **killed** (*requires task object*)
 - **awaited** (*blocking*)
- Using **await**, the coroutine suspends (waits for simulator)
- *Only coroutines can use **await***
- *All methods can schedule tasks using **start_soon()***
- *Tasks that are not awaited will end when simulation finishes*

```
# Coroutine example
async def my_coro(dut)
    await dut.signal.value_change
    <do something>

...

# Non blocking execution
TaskA = start_soon(my_coro(dut))

...

# Optional termination of tasks
TaskA.kill()

...

#Blocking execution
await my_coro(dut)

...

#Termination by coroutine
async def my_killcoro(myTask)
    await <something>
    myTask.kill()

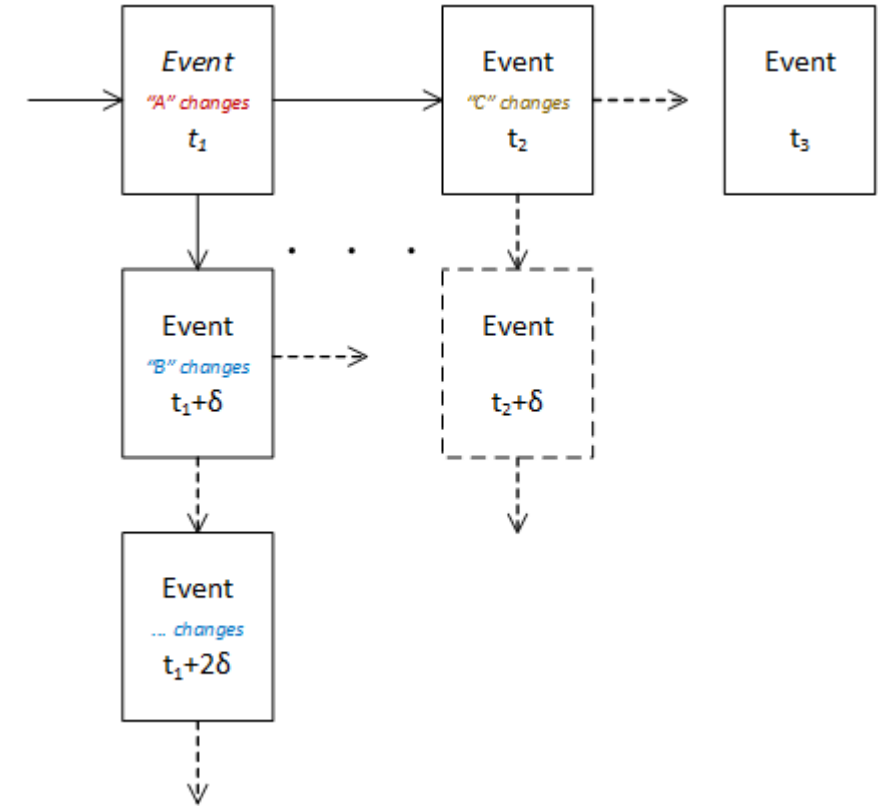
TaskB = my_coro(dut)
start_soon(my_killcoro(TaskB))
await TaskB

...
```

Cosimulation concept

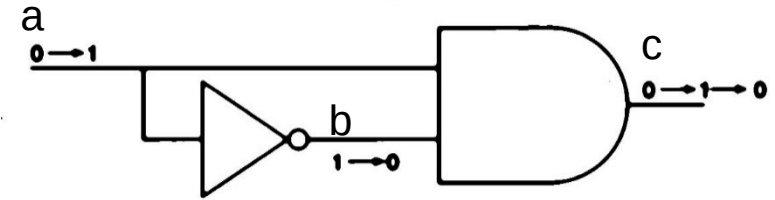
Event driven simulation and ReadOnly phase

- An event occurs in a time-step when a signal changes
 - All signals that depend on the signal is evaluated
 - Consequences are put in the event queue
 - With no timing information (RTL-simulation)
 - Output is queued at the same time-step
 - » A delay of 0 time is called a delta delay (δ)
 - All events in the queue for a time step is evaluated
 - Until there are no more changes left
 - => “**ReadOnly**” phase in cocotb
- The simulation proceeds to the next time-step when there are no more events to be evaluated
- Cocotb can be triggered by *any event* or *timeslot*
 - => *before, mid or after each queue...*



Event queue example : Glitch

Full code (also on github)



```

library IEEE;
use IEEE.std_logic_1164.all;

entity delta is
  port(
    a : in std_logic;
    c : out std_logic
  );
end entity delta;

architecture dataflow of delta is
  signal b : std_logic;
begin
  b <= not a;
  c <= b and a;
end architecture;

import cocotb
from cocotb import start_soon
from cocotb.triggers import Edge, Timer, First
from cocotb.utils import get_sim_time

async def monitor(dut):
  while True:
    ta = dut.a.value_change
    tb = dut.b.value_change
    tc = dut.c.value_change
    await First(ta, tb, tc)
    print(f"{{get_sim_time(units='ps')}}:{{9}}.0f}}ps  ",
          f"a:{{(dut.a.value)}}  ",
          f"b:{{(dut.b.value)}}  ",
          f"c:{{(dut.c.value)}}")

@cocotb.test()
async def main_test(dut):
  dut.a.value = 0
  start_soon(monitor(dut))
  for i in range(5):
    await Timer(100, units='ps')
    dut.a.value = not dut.a.value
  
```

0ps	a:0	b:U	c:U
0ps	a:0	b:1	c:0
100ps	a:1	b:1	c:0
100ps	a:1	b:0	c:1
100ps	a:1	b:0	c:0
200ps	a:0	b:0	c:0
200ps	a:0	b:1	c:0
300ps	a:1	b:1	c:0
300ps	a:1	b:0	c:1
300ps	a:1	b:0	c:0
400ps	a:0	b:0	c:0
400ps	a:0	b:1	c:0

Waiting for ReadOnly()

0ps	a:0	b:1	c:0
100ps	a:1	b:0	c:0
200ps	a:0	b:1	c:0
300ps	a:1	b:0	c:0
400ps	a:0	b:1	c:0

Glitches will typically not be shown in waveforms
- *unless post synthesis simulation*

```
import cocotb
from cocotb import start_soon
from cocotb.triggers import Edge, Timer, First, ReadOnly
from cocotb.utils import get_sim_time

async def monitor(dut):
    while True:
        ta = dut.a.value_change # Edge(dut.a)
        tb = dut.b.value_change # Edge(dut.b)
        tc = dut.c.value_change # Edge(dut.c)
        await First(ta, tb, tc)
        await ReadOnly()
        print(f"{get_sim_time(units='ps'):{9}.0f}ps    ",
              f"a:{{(dut.a.value)}}    ",
              f"b:{{(dut.b.value)}}    ",
              f"c:{{(dut.c.value)}}")

@cocotb.test()
async def main_test(dut):
    dut.a.value = 0
    start_soon(monitor(dut))
    for i in range(5):
        await Timer(100, units='ps')
        dut.a.value = not dut.a.value
```

Cocotb Triggers -- **await** <trigger>

ReadOnly()

Waits until all delta delays has settled
useful for *most* checks

ReadWrite()

Get out of ReadOnly phase

NOTE: *can be a bit sketchy – will wait for next event, whenever that is.*

<signal>.**value_change** **#Edge(signal)** *Waits for any change in the signal*

<signal>.**rising_edge** **#RisingEdge(signal),**

<signal>.**falling_edge** **#FallingEdge(signal)**

Timer(value, unit="ps")

Waits the exact simulation time

ClockCycles(signal, num_cycles, rising=True)

Waits a number of (rising) edges

First(*triggers)

Waits for the first trigger in the list

Combine(*triggers)

Waits for all triggers in the list

How does cocotb use timing information?

How do we define time steps and

- Three major ways of waiting timesteps:
 - `await Timer(..)`
 - `await <signal>.value_change, .rising_edge, .falling_edge`
 - `await Clockcycles(..)`
- Principles of simulation
 - Read goes before write
 - Assignment are put onto the event-queue which resolves “later”.
 - Using the same timer values (*separate methods*) for read and assignment:
 - Assignment will happen later than read.
 - Cocotb.Clock uses Timer()
 - Clk values will never come before **awaiting** Timer()
 - Clockcycles(..) uses rising_edge (or falling_edge when specified)
 - Clockcycles trigger is equivalent to waiting a number of .rising_edge (s)
- => **It is possible to "fool" RTL simulation having signals assigned before clock edges at the same time (Avoid this!)...**

How does cocotb use timing information

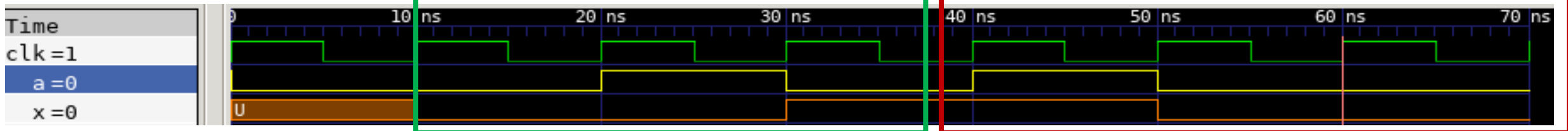
Example

Similar/extended version on

<https://github.com/yngveha/seminars/tree/main/2026/inventas/timing-test>

- Look closely at the register (x) output-
 - How did we get there?

```
x <= a when rising_edge(clk);
```



How does cocotb use timing information

Example

Slightly extended version on
<https://github.com/yngveha/seminars/tree/main/2026/inventas/timing-test>

- VHDL register uses `rising_edge(clk)`
- Stimuli is set using `clk.rising_edge` and `Timer()`
- Values are provided using `Timer()`, `clk.rising_edge` and `Readonly()` :

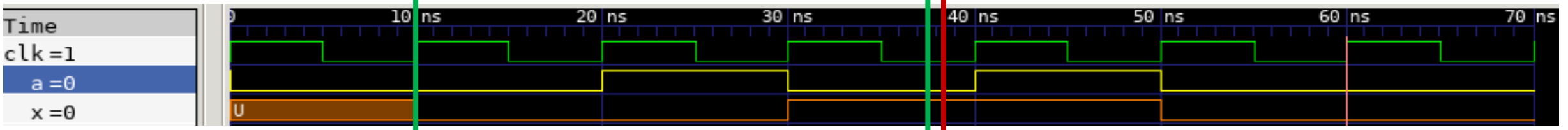
```
x <= a when rising_edge(clk);
```

```
async def Astim(dut):  
    await dut.clk.rising_edge # 20ns  
    dut.a.value = 1  
    await Timer(10, units="ns") # 30 ns  
    await dut.clk.rising_edge # 30 ns  
    dut.a.value = 0  
    await Timer(10, units="ns") # 40 ns  
    dut.a.value = 1  
    await Timer(10, units="ns") # 50 ns  
    dut.a.value = 0
```

20.0ns Awaited RisingEdge, A<=1 after clock	A: 0, x:0 Rising_edge => within the event queue
20.0ns Awaited Readonly()	A: 1, x:0 Readonly => event queue end for the current time
30.0ns Awaited 10ns Timer, A<=0 after clock	A: 1, x:0 Timer => beginning of event queue
30.0ns Awaited RisingEdge	A: 1, x:0 OK, 30 ns means we are in the middle of the event queue
30.0ns Awaited Readonly()	A: 0, x:1 Readonly -> event queue has settled
40.0ns Awaited 10ns Timer, A<='1' after Timed 10ns	A: 0, x:1 As expected, set using timer goes after read using timer
40.0ns Awaited ClockCycle (=Rising edge)	A: 1, x:1 Waiting a clock cycle only waits for rising edge
40.0ns After Readonly()	A: 1, x:1 x should be 0 but isn't because A was ready before clock
50.0ns Awaited 10ns Timer, A<='0' after Timed 10ns	A: 1, x:1 ...
50.0ns Awaited ClockCycle	A: 0, x:1 ...
50.0ns After Readonly()	A: 0, x:0 ...

Expected register behavior

Simulation looks flawed (!)



Timing information in cocotb

Summary for safe use

- Timer intercepts simulation before the event queue *at that timestep*
- *Reading goes before assignment*
- To achieve consistent, comprehensible results:
 - *Apply stimuli after either*
 - `await Clockcycles(dut.clk, #cycles)` *or* `await dut.clk.rising_edge`
 - *Apply checks triggered by*
 - `await <signal>.rising_edge/.falling_edge/.value_change` *followed by*
 - `await ReadOnly()`
 - *Apply Timer only when clock is irrelevant*
 - testing CL or timeout conditions
 - Using Timer + `await ReadOnly()` will work, *but*
 - *hardcoding timing is rarely a good for scalability*
- Timer based stimuli may lead to confusing waveforms
 - *Ad hoc solution* : Having Stimuli delayed compared to clock
 - *Scales poorly*; changing clock period may void testbench
 - *Shifting between timer and Clock relation will lead to inconsistencies*

Cocotb cosimulation

Readability, verifiability and modifiability -
Easily organized?

Called and triggered checks

Queues for data management



UNIVERSITY
OF OSLO



Making use of cocotb

Called versus triggered checks

■ Called checks

```
for data in stimulidata:
    set_stimuli(data)
    called_check(data)
```

- ♦ Must be remembered for every usecase
 - Code coverage $\neq$ data coverage
- ♦ Ties check to specific stimuli and timing
 - *Maintainability can be an issue.*
- ♦ Very little data management required

■ Triggered checks

```
while True:
    await(trigger)
    model_data = stimuli_queue.get()
    assert dut.data.value == model_data
```

- ♦ Always on - *at least through test phase...*
- ♦ Independent of stimuli
 - *Function can be verified any time (FIAT friendly)*
- ♦ Only tied to test specification
 - Easier to maintain
- ♦ Requires data management
 - = *Easy using python queues*



Example testbench - Sensor data interface

Testbench featuring independent stimuli and checks

- Applies predefined test patterns
 - ◆ *Showcases randomized order of execution*
 - Both stimuli and checks are triggered
 - A queue is used for passing data internally
-
- https://github.com/yngveha/seminars/tree/main/2026/inventas/unispi_extended
-
- Show testbench structure
 - ◆ *makefile and vhdl is secondary*
 - Run test



UNIVERSITY
OF OSLO